

$$L(N, D) = 1.87 + \frac{521}{N^{0.353}} + \frac{1488}{D^{0.353}} \quad (20)$$

To verify the accuracy of our fit, we benchmark the predictions with those of the IsoFLOP C4 curves in [42]. Following [42], we can compute the optimal number of parameters N_{opt} and tokens D_{opt} for our fit using:

$$N_{opt}(C) = G\left(\frac{C}{6}\right)^a, \quad D_{opt}(C) = G^{-1}\left(\frac{C}{6}\right)^b \quad (21)$$

where $G = \left(\frac{\alpha A}{\beta B}\right)^{\frac{1}{\alpha+\beta}}$, $a = \frac{\beta}{\alpha + \beta}$, and $b = \frac{\alpha}{\alpha + \beta}$

Given the Gopher compute budget of $C = 5.76 \times 10^{23}$ our fitted parameters predict an optimal allocation of $N_{opt} = 70.0$ billion parameters and $D_{opt} = 1.37$ trillion tokens. This is very close to the 73 billion parameters and 1.3 trillion tokens predicted by the IsoFLOP curves on C4 from [42] and thus we consider it a good fit. We use these fitted parameters rather than the MassiveWeb parameters for all computations involving Chinchilla scaling laws.

C Additional Contour Plots

Figure 8 contains additional empirical isoLoss contours for 400 million and 1.5 billion unique tokens. Results show that like in Figure 3 significantly lower loss can be achieved by increasing parameters and epochs beyond what is compute-optimal at a single epoch. The lowest loss is also achieved by allocating more extra compute to repeating data rather than to adding parameters.

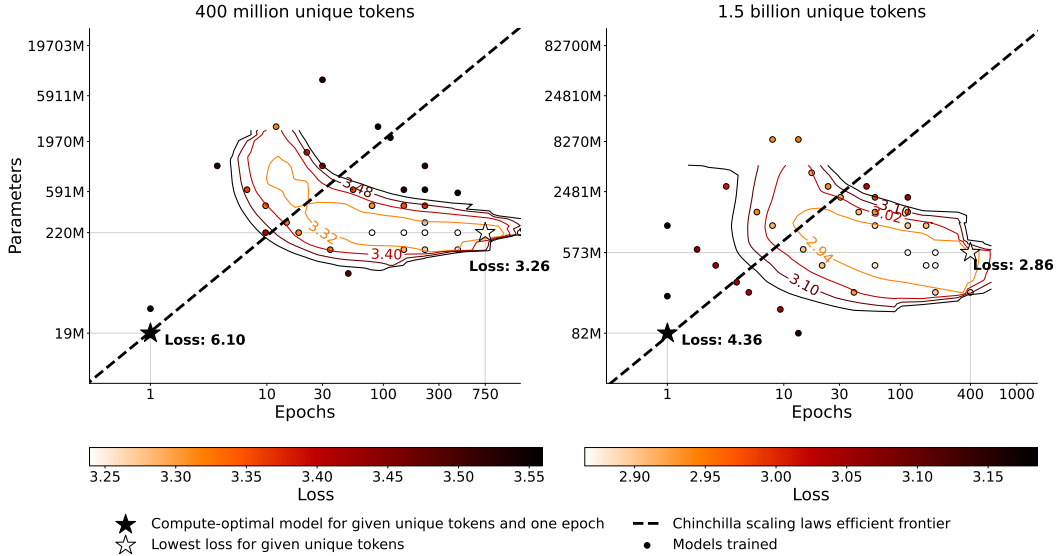


Figure 8: **Empirical isoLoss curves for 400 million and 1.5 billion unique tokens.** 34 models trained on 400 million unique tokens and 37 models trained on 1.5 billion unique tokens with varying parameters and epochs.

D Double Descent

Prior work has reported double descent phenomena when repeating data, where the loss initially increases and then decreases again as the model is trained for more epochs [75, 40]. In Figure 9, we plot the loss curves of several models trained for varying epochs on 100 million tokens. We find double descent phenomena with the loss of all models increasing at 200 epochs before decreasing

again. This contributes to additional noise in the fitting of our functions in [Appendix A](#), as our functional form assumes loss to be monotonically decreasing as epochs increase. Thus, we remove most such examples from the fitting.

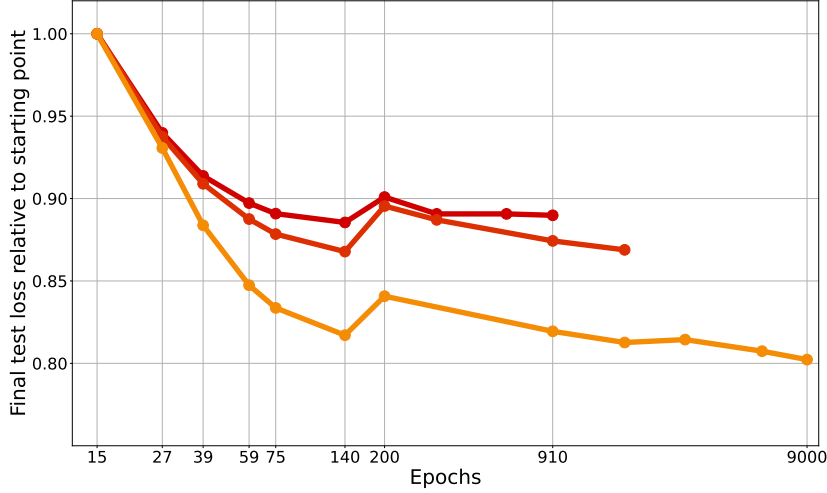


Figure 9: **Double descent.** Each dot is a model trained on 100 million unique tokens. Loss initially increases at 200 epochs and then decreases again; this is known as epoch-wise double descent [75].

E Repeating on Heavily Deduplicated Data

To investigate whether [Figure 3](#) is dependent on the inherent amount of duplicates in the selected 100 million tokens, we train several models on a deduplicated version of C4 (see [Appendix N](#)). We plot the performance of the models trained on the deduplicated C4 versus the regular C4 in [Figure 10](#). All models are evaluated on the same validation dataset from the regular C4. Regardless of deduplication we find 59 epochs to be optimal and the overall trend to be very similar. Together with our results on OSCAR ([Appendix I](#)), this suggests that our work generalizes to different datasets with different inherent amounts of duplicates.

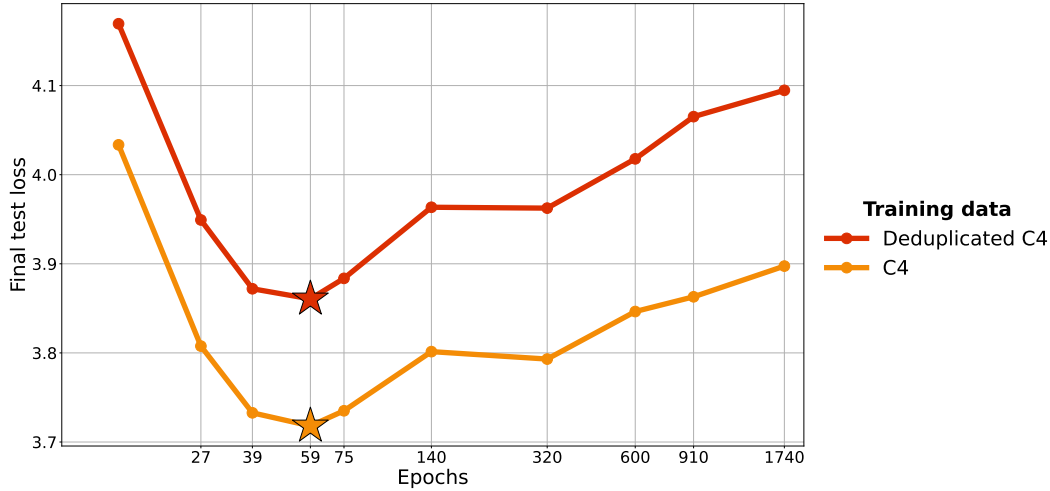


Figure 10: **Optimal loss on deduplicated data.** 146 million parameter models trained on 100 million unique tokens that are either directly from C4 or undergo additional deduplication. Each dot is a single model. While deduplication results in a higher test loss, the optimal number of epochs remains the same whether or not deduplication is performed (see also [Figure 3](#)).

F Do Excess Parameters Hurt, Plateau or Help?

Figures 3, 8 suggest that excess parameters (or epochs) can harm performance. We hypothesize that this is due to suboptimal hyperparameters and could be prevented with better regularization. Thus, we expect with optimal regularization hyperparameters excess parameters would never hurt, but performance would merely plateau, as in extreme cases regularization could just take the form of removing the excess parameters. One approach to selecting optimal hyperparameters is μP [127]. We compare excessively large models trained with a data constraint of $D_C = 100$ million tokens in Figure 11 across μP , our default hyperparameters (Appendix S) and scaling law predictions. Surprisingly, μP leads to even higher test loss than our default hyperparameters. Nevertheless, we find that also with μP excessive parameters hurt: The models with more than 2 billion parameters have significantly higher validation loss after training than the models with 200 million to 1 billion parameters when trained on only 100 million tokens. However, μP only covers hyperparameters such as the learning rate, but not explicit regularization hyperparameters like dropout rates, which we hypothesize would prevent this behavior. Thus, our proposed scaling equations predict loss to plateau, as seen in the straight line. As the compute-optimal parameter count for 100 million tokens is around 7 million, all depicted models have a significant amount of excess parameters and data-constrained scaling laws predict their losses to be all the same ($R_N^* \ll R_N$). Meanwhile, the default Chinchilla scaling law [42] predicts loss to continue decreasing as parameters are added, which is in stark contrast to the empirical data.

If one wants to incorporate excess parameters hurting performance into the scaling law equations, one could consider (a) Modifying the exponential decay formulation introduced in Appendix A such that instead of the value of repeated data decaying to 0 it decays to a large negative value (b) decaying the exponents α and β in Equation 7 instead of D and N . Decaying the exponents to 0 has the effect of more repetitions eventually hurting performance as $\lim_{\alpha \rightarrow 0} D^\alpha = 1$ and the same for β . Thus, initially as D and N increase loss decreases, but ultimately the decay of α and β pushes D and N back to 1 resulting in loss to increase. Specifically, approach (b) could take the form of:

$$L(N, D, R_N, R_D) = E + \frac{A}{N^{\alpha * \max(0, 1 - (R_N / R_N^*))}} + \frac{B}{D^{\beta * \max(0, 1 - (R_D / R_D^*))}} \quad (22)$$

Like the equations in Appendix A this formulation also reduces to the Chinchilla scaling laws in the base case of $R_D = 0$ or $R_N = 0$. As the exponents decrease with more repetitions adding parameters or epochs becomes less beneficial. Eventually, the decay in α or β causes loss to increase again as it pushes N or D back down to 1. We fit this formula using the same approach outlined in Appendix A but including samples where excess parameters or epochs hurt (296 total samples). We use a grid of initialization given by: $R_N^* \in \{0., 2000., \dots, 100000.\}$ and $R_D^* \in \{0., 2000., \dots, 100000.\}$. This results in $R_D^* = 26530.611$ and $R_N^* = 2040.8163$. R_N^* is significantly lower resulting in excess parameters hurting faster than excess epochs, which is in line with empirical data from Figure 3. We visualize Figure 3 with the predictions from this alpha-beta decay formulation in Figure 12. Expected parameters eventually hurt resulting in circle-shaped contours. Due to the very high R_D^* the area where epochs start to hurt is outside of the boundaries of Figure 12. While the predicted optimal allocation (efficient frontier) is similar to Figure 3, the predicted return from repeated data differs significantly. The alpha-beta decay formulation incorrectly predicts returns to diminish significantly slower as seen by the longer efficient frontier and the smaller distance in contours early on as compared to Figure 3. Beyond its potentially useful properties, we do not have a rigorous mathematical justification for this alpha-beta decay formulation which could be the cause of the incorrect return predictions.

Ultimately, we settle on our exponential decay formulation from Appendix A that does not allow excess parameters or epochs to hurt, as preventing such behavior is trivial by stopping training (in the case of epochs hurting) or removing excess parameters (in the case of model parameters hurting). Further, accurately predicting how much loss increases in the limit is not very useful, as in practice one would want to stop training when it's expected to plateau anyways.